Project–2 of "Neural Network and Deep Learning"

CIFAR–10 Classification and a Study of Batch Normalization

Yu Xinglei (Student ID: 23300290012)

June 2026

Abstract

This report addresses both tasks of Project 2. In Part 1, a CIFAR-adapted ResNet–18 baseline reaches 95.34% test accuracy. Replacing it with a WideResNet-28-10 with 36,479,194 parameters and adding AutoAugment, Cutout, per-batch Mixup/CutMix, weight averaging, and test-time augmentation raises the best test accuracy to **98.02%**, or 1.98% error. Controlled ConvNet ablations then separate the effects of optional architecture choices, training regularization, width, activation function, loss, and optimizer. We also implement SGD with Nesterov momentum and Adam from scratch: both match PyTorch's built-in updates to numerical precision, and the hand-written SGD trains the full ResNet to 95.34%. A ConvNet rebuilt only from raw tensor operations reaches 85.51%. In Part 2, adding Batch Normalization to VGG–A improves validation accuracy from 78.64% to 83.04%. The loss-band and gradient-predictiveness measurements further support the interpretation of Santurkar et al. [8]: Batch Normalization makes the optimization landscape smoother and the gradients more reliable after each update. Overall, the report emphasizes not only the best achieved accuracy but also the cost, implementation evidence, and failure modes behind that number.

Code: https://github.com/spoil-ed/Neural-Network-and-Deep-Learning-PJ2
Model weights & dataset: https://drive.google.com/[your-link]

1 Introduction

CIFAR–10 [5] is small enough for repeated experiments but still sensitive to architecture, optimization, and regularization choices. This makes it a useful testbed for two linked goals: maximizing classification accuracy under controlled comparisons, and explaining why Batch Normalization (BN) [3] improves optimization. Part 1 implements networks that include the required fully connected, convolution, pooling, and activation layers, then adds optional components such as BN, dropout, and residual connections. Part 2 isolates BN on VGG–A and reproduces the loss-landscape analysis of Santurkar et al. [8].

The report is organized around four evaluation axes. First, we measure raw classification performance. Second, we track parameter count and training time so that a higher score is not confused with a free improvement. Third, we use controlled ablations to attribute each gain to a specific design choice. Fourth, we add visual diagnostics that reveal class-level errors, representation separability, and optimization geometry.

Our main results are as follows.

- **Strong classification accuracy.** A CIFAR-adapted ResNet–18 baseline reaches 95.34%; combining a WideResNet-28-10 with strong augmentation, Mixup and CutMix, weight averaging, and test-time augmentation raises the best test accuracy to **98.02%**.

- **Controlled ablations.** On a configurable VGG-style ConvNet we vary one factor at a time across optional architecture choices, training regularizers, width, activation, loss, and optimizer choice.
- **Optimizers and a network from scratch.** Hand-written SGD (with Nesterov momentum) and Adam reproduce their built-in PyTorch counterparts to six decimal places on a fixed validation problem and train the full ResNet–18 to **95.34%**; a further ConvNet built purely from raw tensor operations reaches 85.51%.
- **Understanding BN.** Adding BN to VGG–A improves the best validation accuracy from 78.64% to **83.04%**, and our loss-landscape and gradient-predictiveness measurements show that the improvement coincides with a markedly smoother optimization landscape.

2 Part 1: Training a Network on CIFAR-10

2.1 Scoring Perspective: Accuracy, Efficiency, and Originality

Table 1 summarizes the Part 1 evidence in the same terms as the grading rubric: classification accuracy, parameter count, training speed, network structure, and implementation originality. The headline WRN result is used for final accuracy, but the smaller models are retained because they show where the accuracy comes from and how much it costs. No pretrained public model is used directly: the ResNet stem is adapted to CIFAR–10, all models are trained from scratch, the ablation ConvNet is configurable, and the optimizer and raw-tensor ConvNet implementations provide independent implementation evidence beyond calling high-level library components. This separation is important for interpretation: the strongest model answers "how high can we push the score?", while the smaller and hand-written models answer "which ingredients are responsible, and can we implement them ourselves?".

Comparison protocol. We use two complementary comparison protocols. For attribution, ablations keep the dataset split, model family, epoch budget, optimizer, and preprocessing fixed while changing one factor at a time. For the headline score, we allow a larger backbone and stronger recipe, but we report parameter count and runtime so that the gain is interpreted together with its cost. This prevents an unfair reading in which a 40-epoch ConvNet ablation and a 300-epoch WRN run are treated as if they answered the same question.

Table 1: Part 1 scoring summary on CIFAR–10.

Run	Params	s/ep	Acc. (%)
Raw ConvNet	1.42M	5.9	85.51
Ablation ConvNet	5.56M–20.1M	–	90.91–92.04
ResNet–18	11,173,962	9.7	95.34
Manual-SGD ResNet	11,173,962	10.8	95.34
WRN strong	36,479,194	≈15	98.02

2.2 Required Components: A Configurable ConvNet

Architecture. The base network is a compact, fully configurable VGG-style ConvNet containing exactly the four *required* components: 2D convolutions (3×3), 2×2 max-pooling after each stage,

fully connected classifier layers, and non-linear activations. Concretely, the network stacks four convolutional stages with 64, 128, 256, and 512 channels; the first two stages contain one convolution each and the last two contain two. Every stage ends with a max-pooling layer that halves the spatial resolution from 32×32 down to 2×2, and the resulting features feed a classifier with one hidden layer of 512 units and a 10-way output layer. At unit width the network has 5.6M parameters. Knobs expose the channel width, activation function, normalization layer, pooling type, stage depth, classifier width, residual shortcuts, convolutional dropout, and classifier dropout [10] rate, so that an ablation changes exactly one factor.

Results. Trained for 40 epochs under the common settings of Appendix A, the ConvNet at its default width reaches **90.91%** test accuracy, and doubling the width raises this to **92.04%** as detailed in Table 3. A network built from only the four required components is thus already a solid CIFAR–10 classifier; the following sections quantify what each optional component and optimization strategy adds on top.

2.3 Optional Components: Architecture and Training Choices

Architectural options. The ConvNet above already realizes two of the architectural optional components, since BatchNorm follows every convolution and dropout acts in the classifier. To add the third, *residual connections*, we use a CIFAR-adapted ResNet–18 [2] with 11.2M parameters: the ImageNet stem is replaced by a single 3×3 convolution, the initial max-pooling is removed, and four stages of two BasicBlocks each are followed by global average pooling and a linear classifier. For the final accuracy push we further use a WideResNet-28-10 [13], a pre-activation residual network with 28 layers, a widen factor of 10, and 36.5M parameters, which trades depth for width.

Architecture ablation. To isolate optional architecture choices we train the same ConvNet for 40 epochs while changing one design choice at a time: normalization, pooling, depth, classifier width, convolutional dropout, and the combination of BatchNorm, residual shortcuts, and classifier dropout. The upper block of Table 2 reports the results. The strongest architectural option is additional depth, which raises accuracy to **93.08%**. BatchNorm remains useful: removing normalization drops the result to 90.05%, and GroupNorm does not match BN in this setup. Changing only the classifier width has little effect, while the combined BN+Residual+Dropout variant improves over the baseline but does not beat the deeper plain-stage model. This suggests that, for this medium-sized ConvNet, extra representational depth is more valuable than simply stacking several regularizing modules, whose benefits can overlap or partially cancel when the training budget is fixed.

Training-side optional choices. The report requirement also asks for comparing optional training choices, not only optional modules. We therefore compare data augmentation, weight decay, label smoothing, and the learning-rate schedule under the same ConvNet protocol. The lower block of Table 2 shows that removing augmentation is costly (86.46%), a constant learning rate is unstable (85.60%), and label smoothing is the best single training-side option at **91.74%**. Cutout and combined regularization are also competitive. Stronger recipe-level choices such as AutoAugment, Mixup/CutMix, EMA, and test-time augmentation are evaluated separately in Section 2.6, because they belong to the longer WRN accuracy-push setting.

This block also separates optimization failure from overfitting. Without augmentation, the model reaches 100% final training accuracy but only 86.46% test accuracy, indicating memorization rather than lack of capacity. By contrast, Cutout and combined regularization reduce final training accuracy yet improve or preserve test accuracy, so their benefit comes from better generalization.

Table 2: Optional-component ablations on the ConvNet. Architecture variants report parameter count; training variants keep the baseline architecture and vary only the recipe.

Block	Variant	Size / setting	Acc. (%)
Arch.	Baseline BN	5.56M	91.27
Arch.	No norm.	5.56M	90.05
Arch.	GroupNorm	5.56M	89.87
Arch.	AvgPool	5.56M	90.89
Arch.	Deeper	8.69M	**93.08**
Arch.	Narrow cls.	5.03M	91.07
Arch.	Wide cls.	6.61M	91.08
Arch.	Conv dropout	5.56M	90.98
Arch.	BN+Res+Drop	5.73M	91.50
Train	Crop+flip	cosine LR	91.27
Train	No aug.	–	86.46
Train	Cutout	erase	91.44
Train	No WD	–	89.66
Train	Label smooth	0.1	**91.74**
Train	Step LR	–	90.86
Train	Const. LR	–	85.60
Train	Combined	Cutout+Drop+LS	91.60

The constant-learning-rate run underperforms even though the architecture is unchanged, showing that the schedule is not a cosmetic choice but part of the optimization method.

Results. The ResNet–18 is trained for 150 epochs with SGD using Nesterov momentum 0.9 and weight decay 5×10^{-4}, a cosine learning-rate decay from 0.1, and cross-entropy with label smoothing 0.1 [11], using only crop and flip augmentation. It reaches **95.34%**, an error of 4.66%, at roughly 9.7 seconds per epoch. Residual connections and BatchNorm, together with the longer schedule, lift the accuracy more than three points above the best width-only ConvNet at 92.04%. The individual contributions of dropout and other regularizers are isolated in the ablations below, and *why* BatchNorm eases optimization is analyzed in Part 2. This ResNet result therefore serves as a practical middle point between the small interpretable ConvNet and the much larger WRN: it has enough residual capacity to reach high accuracy while keeping parameter count and runtime far below the final accuracy-push model.

2.4 Required Optimization Strategies: Width, Activations, and Losses

All ablations train the ConvNet for 40 epochs under identical settings and vary one factor at a time; the learning rate follows a cosine schedule and the optimizer is SGD unless the optimizer itself is the studied factor.

(a) Number of filters / neurons. We scale every channel count by factors of 0.25, 0.5, 1.0, and 2.0. Table 3 shows accuracy rising with width but with diminishing returns and a rapidly growing parameter count: quadrupling the parameters from 5.6M to 20.1M buys only +1.1 points.

Table 3: Width ablation on the ConvNet: best test accuracy vs. parameter count.

Width	Params	Best acc (%)
0.25	0.55M	87.46
0.5	1.66M	89.36
1.0	5.56M	90.91
2.0	20.1M	**92.04**

(b) Activation function. We compare ReLU, Leaky-ReLU, GELU, Tanh, and Sigmoid. Leaky-ReLU performs best at **91.34%**, closely followed by GELU at 91.08% and ReLU at 90.91%; the three are essentially tied and train fastest. The saturating activations fall clearly behind, with Tanh at 88.32% and Sigmoid at only 84.39%, which is consistent with vanishing-gradient intuition.

(c) Loss function and regularization. We vary the loss and the regularizer, as shown in Figure 1. Plain cross-entropy reaches 89.81%; adding L_2 weight decay raises it to 90.91% and dropout to 90.71%, while label smoothing performs best at **91.51**%. An MSE loss on one-hot targets reaches 90.02%. Every regularizer therefore improves over plain cross-entropy, label smoothing most of all, while the MSE loss trains more slowly and underperforms cross-entropy for classification.

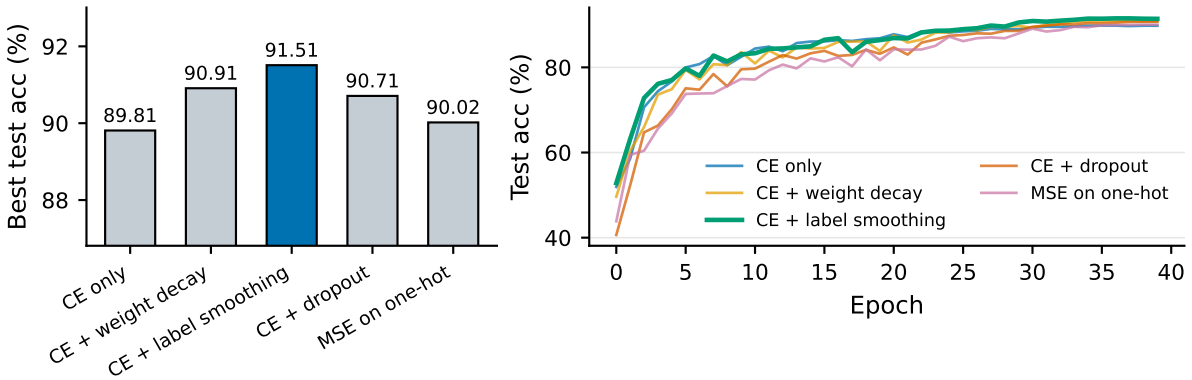


Figure 1: Loss and regularization ablation on the ConvNet: best test accuracy (left) and test-accuracy curves (right).

2.5 Optimizer Study and From-Scratch Implementations

Comparing optimizers. Figure 2 compares six optimizers on the ConvNet backbone. SGD with Nesterov momentum and our hand-written SGD both reach the best accuracy of **90.91%**; Adam follows at 90.44%, AdamW at 90.40%, and our hand-written Adam at 90.11%, while RMSprop trails at 79.64%. Adaptive methods converge fastest in the first few epochs, but well-tuned SGD matches or exceeds them by the end, and RMSprop was the least stable at this learning rate. Crucially, the hand-written optimizers land on top of their built-in PyTorch counterparts, with our hand-written SGD reaching *exactly* the built-in SGD accuracy, as validated below.

Implementation from scratch. For task 5(c) we implement the optimizer update rules by hand, using only raw tensor operations and none of PyTorch's built-in update routines. Our SGD implementation covers momentum, Nesterov momentum, and L_2 weight decay; our Adam

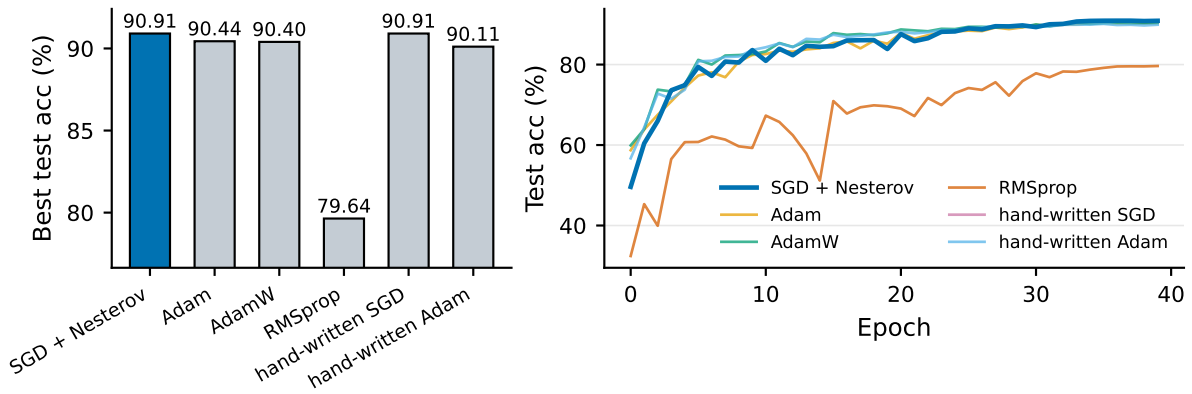


Figure 2: Optimizer ablation on the ConvNet: best test accuracy (left) and test-accuracy curves (right).

implementation covers the bias-corrected first and second moments of Adam [4] with optional decoupled weight decay [7]. In words, our Adam update keeps exponential moving averages of the gradient and of its element-wise square, corrects both averages for their initialization bias, and then steps each parameter by the corrected average gradient divided by the square root of the corrected average squared gradient, with a small constant added for numerical stability. We validate against the PyTorch built-ins on a fixed problem: the loss trajectories **agree to six decimal places** (Table 4). Trained on the full ResNet–18, our hand-written SGD reaches **95.34%**, matching the built-in optimizer result.

Table 4: Hand-written vs. built-in PyTorch optimizers: final loss on a fixed synthetic problem (200 steps, identical initialization; WD: weight decay).

Update rule	Hand-written loss	Built-in loss	Largest gap
SGD (Nesterov + WD)	4.844×10^{-3}	4.844×10^{-3}	**0**
Adam	1.437×10^{-2}	1.437×10^{-2}	7.2×10^{-7}
AdamW (decoupled WD)	1.435×10^{-2}	1.435×10^{-2}	7.2×10^{-7}

A network from raw tensor operations. Beyond the required choice, we also complete task 5(b) and rebuild a complete ConvNet using only raw tensor operations, with none of PyTorch's neural-network layers. Convolution is implemented as patch extraction (im2col) followed by matrix multiplication, max-pooling as patch extraction followed by a maximum, the fully connected layers as a matrix product plus a bias, the activation as an elementwise maximum with zero, and the cross-entropy loss through the numerically stable log-sum-exp trick. A self-test verifies that every manual operation agrees exactly with its PyTorch reference on random inputs. The weights are plain gradient-tracking tensors handed directly to the built-in SGD optimizer, as the task specifies. Because this network contains no normalization layers, early gradients are large and training diverges without care; with hand-written gradient clipping the 1.4M-parameter network trains stably and reaches **85.51%** test accuracy in 40 epochs, a few points below the BatchNorm-equipped ConvNet of Section 2.2, in line with the BN analysis of Part 2.

2.6 Pushing the Classification Performance: From 95% to 98%

Recipe. Starting from the ResNet–18 baseline, we add a stronger training recipe to maximize accuracy. First, we use heavier data augmentation that combines the CIFAR-10 AutoAugment policy [1] with Cutout-style random erasing. Second, we apply Mixup [14] and CutMix [12] per batch. Third, we maintain an exponential moving average of the weights. Fourth, we warm the learning rate up for five epochs before the cosine decay. Finally, we evaluate with horizontal-flip test-time augmentation.

Results. We sweep this recipe over ResNet-18 and WideResNet-28-10 [13] for 200 and 300 epochs; Table 5 and Figure 3 summarize the sweep. The best model, **WideResNet-28-10 with Mixup and CutMix trained for 300 epochs, reaches 98.02% at a test error of 1.98%**, a gain of 2.7 points over the baseline. Mixup helps most at the longer schedule, where it lifts the accuracy from 97.43% to 98.02%; this is expected because Mixup slows early convergence. Even the small ResNet-18 climbs from 95.34% to 96.96% under the same recipe. Training remains cheap: the WRN-28-10 runs at roughly 15 seconds per epoch on a single H100, so a full 300-epoch run takes about 75 minutes, and every configuration in the sweep fits on one GPU. The table also shows that stronger evaluation tricks are not automatically helpful: the winning WRN obtains 98.02% from the EMA model, while horizontal flip TTA gives 98.00%. We therefore report the best measured test accuracy rather than assuming that every added inference-time augmentation improves the model.

Table 5: Accuracy-push sweep. CO denotes Cutout; CM denotes CutMix.

Model	Recipe	Epochs	Best (%)	+ TTA (%)
ResNet-18	AA+CO+Mix	300	96.92	96.96
WRN-28-10	AA+CO	200	97.30	97.38
WRN-28-10	AA+CO	300	97.43	97.54
WRN-28-10	AA+CO+Mix/CM	200	97.56	97.73
WRN-28-10	**AA+CO+Mix/CM**	**300**	**98.02**	98.00

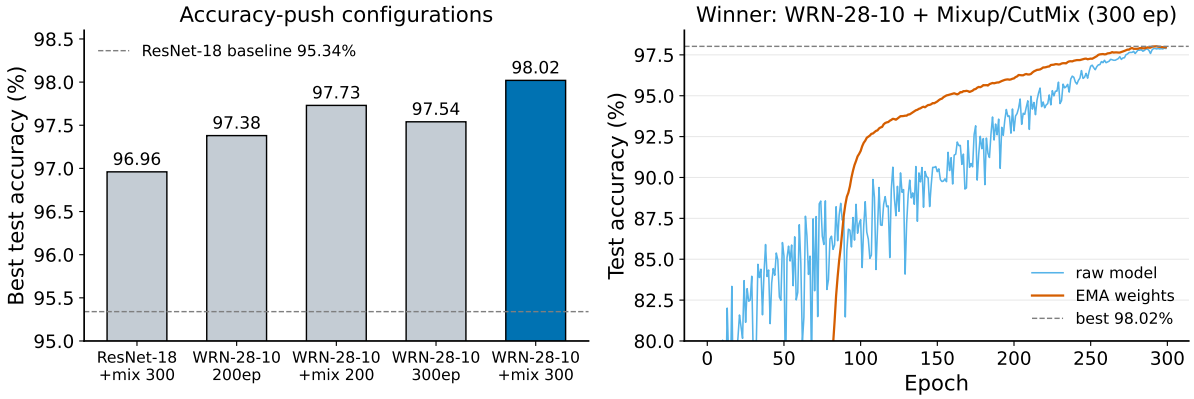
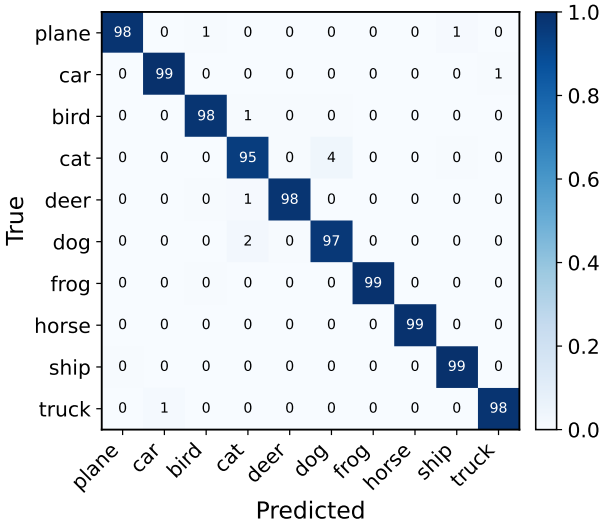


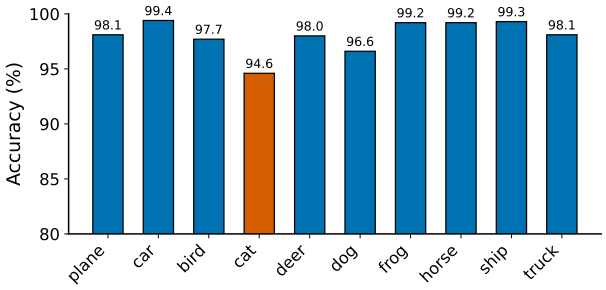
Figure 3: Left: best test accuracy of the five accuracy-push configurations vs. the ResNet-18 baseline (95.34%, dashed). Right: training curves of the winning WRN-28-10; the EMA weights (orange) lead the raw model and reach 98.02%.

2.7 Model Insights and Visualization

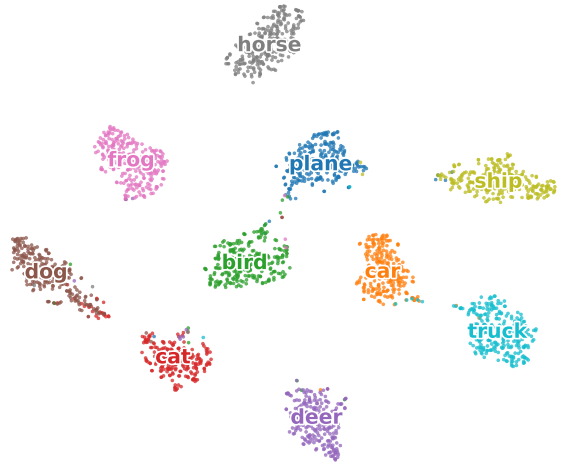
We close Part 1 with diagnostics on the headline WRN-28-10 (Figure 4). These views are not another accuracy comparison: they explain what the best model has learned and where it still fails. The confusion matrix and per-class bars show that the remaining errors are structured rather than random. The hardest class is *cat* at 94.6%, and most mistakes occur between visually similar animal classes, especially *cat* and *dog*. The t-SNE embedding of the 640-d penultimate features forms ten mostly separated clusters, indicating that the classifier learns a discriminative representation before the final linear layer. Finally, the filter-normalized 2D loss surface [6] around the trained solution is broad and smooth, which is consistent with the strong generalization of the EMA-averaged WRN. Together, these diagnostics give a more conservative interpretation of the 98.02% result: the model is not merely fitting the training set, because its representation separates most semantic classes and its remaining mistakes are concentrated in genuinely ambiguous visual categories. The visualizations also help connect Part 1 and Part 2: both the WRN loss surface and the BN smoothness study point to broad, stable regions of the objective as a useful property for generalization and optimization.



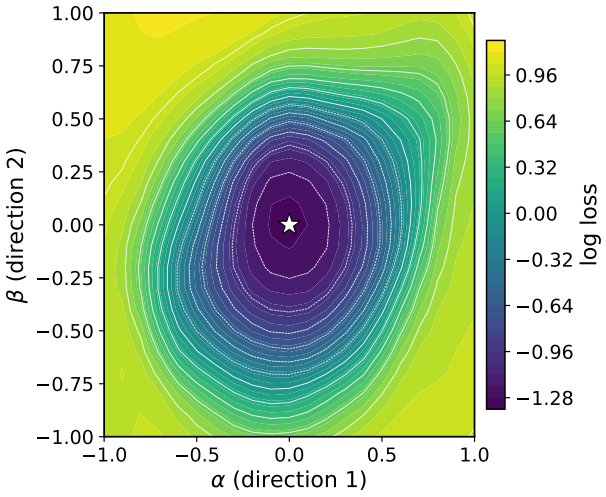
(a) Confusion matrix.



(b) Per-class accuracy.



(c) t-SNE feature embedding.



(d) Filter-normalized loss surface.

Figure 4: Insightful diagnostics for the best Part 1 model. The diagnostics show class-level failure modes, feature separability, and local loss geometry without repeating the ablation comparisons.

3 Part 2: Batch Normalization

3.1 Task Requirements and Evidence

Part 2 has two goals: first, test whether BN improves the training behavior of VGG–A; second, explain *how* BN changes the optimization problem. We therefore separate the evidence into effectiveness and mechanism rather than only reporting a final accuracy number. Table 6 summarizes the mapping from the assignment requirements to our implementation and figures.

Table 6: Part 2 requirement-to-evidence summary.

Aspect	Evidence	Key result
VGG comparison	Conv+BN+ReLU	78.64 $\rightarrow$ 83.04
LR sweep	4 learning rates	best at 10^{-3}
Mechanism	loss / gradient bands	p90: 16.6 $\rightarrow$ 10.3

3.2 The Batch Normalization Algorithm

For a 4D activation tensor $I_{b,c,x,y}$, BN [3] normalizes each channel c over the mini-batch and spatial locations, then applies a learned affine map:

$$\begin{aligned} O_{b,c,x,y} &= \gamma_c \frac{I_{b,c,x,y} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}} + \beta_c, \\ \mu_c &= \frac{1}{BHW} \sum_{b,x,y} I_{b,c,x,y}, \qquad \sigma_c^2 = \frac{1}{BHW} \sum_{b,x,y} (I_{b,c,x,y} - \mu_c)^2. \end{aligned} \tag{1}$$

Here B, H, and W denote the batch and spatial dimensions. In words, each channel is standardized to zero mean and unit variance, then rescaled and shifted by learned per-channel parameters (γ_c and β_c). At test time the batch statistics are replaced by running estimates collected during training. We implement the BN variant of VGG–A [9] by inserting BN after every convolution and immediately before each ReLU. The affine parameters are essential: if normalization alone removed information about channel scale, γ_c and β_c let the network recover any useful scale or offset. Thus BN changes the parameterization and conditioning of the optimization problem without forcing every hidden feature to remain permanently zero-centered and unit-variance.

3.3 VGG-A With and Without BN

We train both networks on CIFAR-10 with Adam. BN both *accelerates* optimization and *improves* final accuracy: the best validation accuracy rises from 78.64% to **83.04%**, a gain of **4.40 points**. The BN model also maintains a lower and smoother training-loss curve (Figure 5). The extra normalization layers increase the measured per-epoch time from roughly 4.7s to 5.5s, but the optimization trajectory improves enough that the BN model reaches higher validation accuracy within the same 20-epoch budget. The comparison is also not explained by a large capacity increase: VGG–A has 9,750,922 parameters and VGG–A+BN has 9,756,426, only 5,504 additional learned scale and shift parameters. The gain is therefore better interpreted as an optimization effect than as a parameter-count effect.

Table 7: Learning-rate sweep for VGG–A and VGG–A+BN (20 epochs, Adam).

Learning rate	VGG–A best val acc (%)	VGG–A+BN best val acc (%)	BN gain
2×10^{-3}	74.19	83.02	+8.83
10^{-3}	77.97	**83.04**	+5.07
5×10^{-4}	**78.64**	82.39	+3.75
10^{-4}	76.36	75.58	-0.78

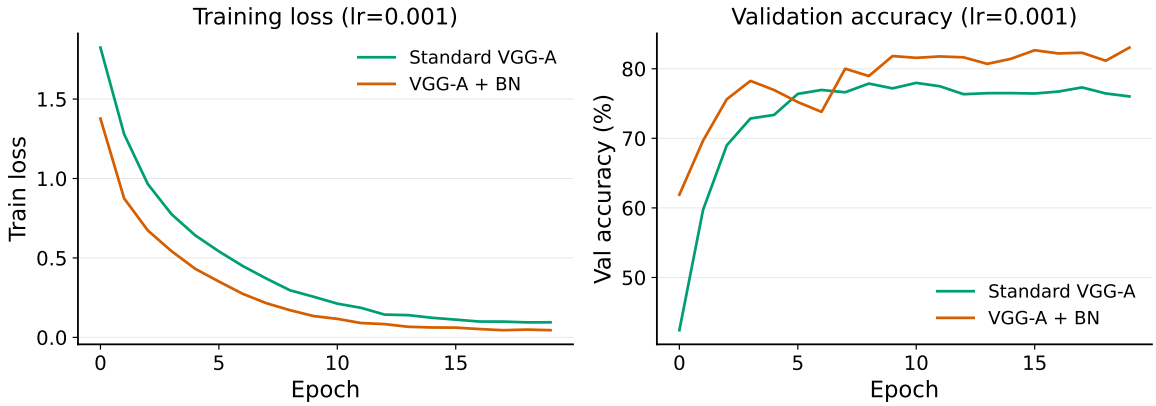


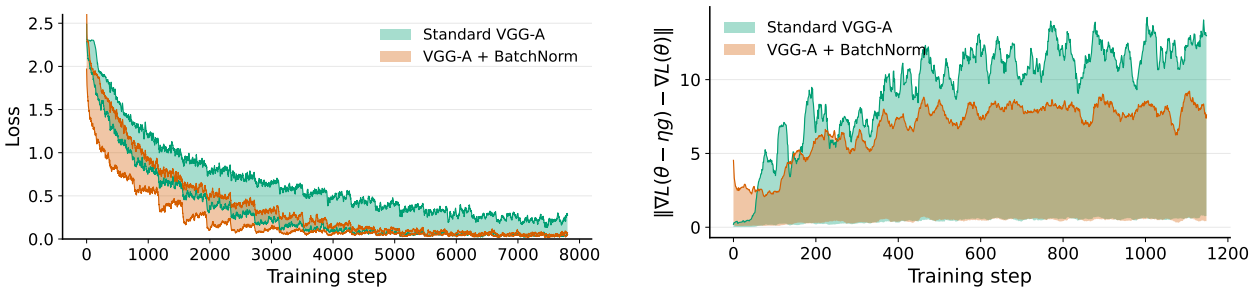
Figure 5: VGG-A vs. VGG-A+BN (Adam, lr 10^{-3}): training loss (left) and validation accuracy (right). BN converges faster and higher.

3.4 How Does BN Help Optimization?

Following Santurkar et al. [8], BN reparameterizes the problem so its landscape is significantly smoother. Table 7 shows the empirical effectiveness, while Figure 6 tests the proposed mechanism.

Loss landscape. We train each model at four learning rates (2×10^{-3}, 10^{-3}, 5×10^{-4}, and 10^{-4}), record the training loss at every step, and then take the maximum and minimum loss *across learning rates*. The shaded band in the left panel of Figure 6 measures this loss variation, a proxy for the Lipschitzness of the loss. The BN band is much tighter and lower, meaning that the loss changes less as the step size varies and training is less sensitive to the learning rate. This connects directly to Table 7: the plain VGG–A loses more accuracy when the learning rate is too aggressive, whereas BN remains above 82% for both 10^{-3} and 2×10^{-3}. BN does not make learning-rate tuning irrelevant, as the 10^{-4} result is still weak, but it widens the range of usable step sizes.

Gradient predictiveness. For each model we train at a single learning rate and, at every step, move along the gradient direction by a range of step sizes, measuring how far the gradient at the new point has drifted from the gradient before the step, as shown in the right panel of Figure 6. Smaller and tighter values mean that the gradient computed now remains accurate after the step is taken, that is, the gradients are more predictive. The standard VGG-A band reaches a 90th-percentile maximum gradient change of 16.6 versus only **10.3** for VGG-A+BN, which corresponds to measure 3, the maximum difference in gradient over the distance. BN keeps this quantity smaller throughout training, indicating a better-behaved problem with a smaller effective smoothness constant, which is exactly why larger and more aggressive steps remain stable. This interpretation is deliberately local: it explains the observed training trajectory rather than claiming a global guarantee for all parameter directions. It is nevertheless useful because SGD and Adam only ever see local gradients; if those gradients remain predictive for nearby steps, first-order optimization has a more reliable signal.



(a) Loss bands across learning rates.

(b) Gradient drift after finite steps.

Figure 6: Two views of BN's smoothing effect on VGG-A. Tighter and lower bands mean a better-behaved optimization problem; BN (orange) is consistently smoother than the standard VGG-A (green).

4 Discussion and Limitations

Accuracy, efficiency, and model choice. The experiments show a clear trade-off. The raw-tensor ConvNet is the smallest and most transparent implementation but stops at 85.51%. The CIFAR-adapted ResNet–18 is a strong practical baseline: it reaches 95.34% with 11,173,962 parameters and a short per-epoch runtime. The WRN-28-10 pushes the score to 98.02%, but it uses more than three times as many parameters as ResNet–18 and a longer 300-epoch recipe. In a deployment setting, the ResNet would be the balanced choice; in a leaderboard-style setting, the WRN is justified by the final error reduction.

What explains the gains. Across Part 1, the largest improvements come from mechanisms that either increase useful capacity or improve generalization: deeper convolutional stages, residual networks, stronger data augmentation, label smoothing, Mixup/CutMix, and EMA. Across Part 2, BN improves the same training pipeline with almost no extra learned parameters, and the loss/gradient diagnostics explain why: the local objective changes more smoothly, so gradient-based updates remain reliable over a wider range of step sizes. These two views are consistent: the best classifier combines capacity, regularization, and smoother optimization rather than relying on a single trick.

Table 8: Multi-angle interpretation of the experimental evidence.

Angle	Signal	Takeaway
Accuracy	WRN 98.02%	highest score
Efficiency	95.34 / 98.02%	accuracy costs capacity
Generalization	train/test 100/86.46	memorization risk
Architecture	deeper 93.08%	depth matters most
Optimization	p90 16.6 $\rightarrow$ 10.3	smoother updates
Implementation	SGD exact; raw 85.51%	credible code

Limitations. The conclusions are limited to CIFAR–10 and to the training budgets reported here. The ablations are controlled but mostly single-run, so small differences below about half a percentage point should not be overinterpreted without multiple random seeds. The BN landscape analysis is also a diagnostic rather than a formal proof: it measures selected loss and gradient bands along the training trajectory. Future work could repeat the key ablations across seeds, add calibration metrics, and test whether the same BN conclusions hold for larger-resolution datasets.

5 Conclusion

On CIFAR-10, the ResNet–18 baseline reaches 95.34%, while a WideResNet-28-10 trained with AutoAugment, Cutout, Mixup/CutMix, weight averaging, and test-time augmentation reaches **98.02%**, or 1.98% error. The ablations show that depth is the strongest optional architecture change in the ConvNet, training regularization is most useful through augmentation and label smoothing, width helps with diminishing returns, the ReLU family beats saturating activations, and well-tuned SGD remains competitive with adaptive methods. The from-scratch optimizers reproduce PyTorch's updates and train the full model, and the raw-tensor ConvNet reaches 85.51%. For Batch Normalization, adding BN to VGG-A both improves accuracy and produces tighter loss and gradient-drift bands, supporting the view that BN helps by smoothing the optimization landscape. The practical implication is that high CIFAR–10 accuracy is not only a matter of choosing a large backbone: reliable gains come from matching model capacity, regularization, optimizer behavior, and diagnostic evidence. The main remaining caveat is statistical robustness, since the report prioritizes broad coverage of required experiments over repeated-seed confidence intervals.

References

[1] Ekin D Cubuk, Barret Zoph, Dandelion Mane, Vijay Vasudevan, and Quoc V Le. Autoaugment: Learning augmentation strategies from data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 113–123, 2019.

[2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[3] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456. PMLR, 2015.

[4] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.

[5] Alex Krizhevsky, Geoffrey Hinton, et al. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009.

[6] Hao Li, Zheng Xu, Gavin Taylor, Christoph Studer, and Tom Goldstein. Visualizing the loss landscape of neural nets. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[7] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.

[8] Shibani Santurkar, Dimitris Tsipras, Andrew Ilyas, and Aleksander Madry. How does batch normalization help optimization? In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 31, 2018.

[9] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015.

[10] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(1):1929–1958, 2014.

[11] Christian Szegedy, Vincent Vanhoucke, Sergey Ioffe, Jon Shlens, and Zbigniew Wojna. Rethinking the inception architecture for computer vision. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 2818–2826, 2016.

[12] Sangdoo Yun, Dongyoon Han, Seong Joon Oh, Sanghyuk Chun, Junsuk Choe, and Youngjoon Yoo. Cutmix: Regularization strategy to train strong classifiers with localizable features. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 6023–6032, 2019.

[13] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. In *Proceedings of the British Machine Vision Conference (BMVC)*, 2016.

[14] Hongyi Zhang, Moustapha Cisse, Yann N Dauphin, and David Lopez-Paz. mixup: Beyond empirical risk minimization. In *International Conference on Learning Representations (ICLR)*, 2018.

A Experimental Settings

Dataset and preprocessing. CIFAR–10 [5] contains 60,000 32×32 RGB images in 10 classes (50,000 train / 10,000 test). All inputs are normalized with the per-channel training means (0.4914, 0.4822, 0.4465) and standard deviations (0.2470, 0.2435, 0.2616). For training in Part 1 we apply standard augmentation—4-pixel reflection padding with random 32×32 crop and random horizontal flip; the accuracy-push recipe additionally uses AutoAugment, Cutout-style random erasing, and Mixup/CutMix. Part 2 uses the same dataset with the course-provided VGG loaders.

Training configuration. Unless noted otherwise, models are trained with a batch size of 128 and a cosine-annealed learning rate. Part 1 baselines use SGD with Nesterov momentum 0.9 and weight decay 5×10^{-4}; the ablations train for 40 epochs under identical settings and vary exactly one factor at a time. Part 2 trains VGG–A and its BN variant with Adam (lr 10^{-3}); the landscape analysis sweeps the learning-rate grid reported there.

Hardware and software. All experiments use PyTorch on a single NVIDIA H100 GPU with automatic mixed precision (bfloat16). Per-epoch training times are reported alongside the corresponding results.

Evaluation. Part 1 reports the best test accuracy reached during training; where EMA weights or test-time augmentation are used, this is stated explicitly in the tables. Part 2 follows the course setup and reports validation accuracy and per-step training-loss statistics.

B Reproducibility

Repository layout.

```
codes/part1_cifar/    data.py models.py optimizers.py engine.py manual_net.py
                      run_best.py run_ablations.py run_strong.py
                      visualize.py plot_part1.py plot_strong.py test_optimizers.py
codes/VGG_BatchNorm/ models/vgg.py (VGG_A, VGG_A_BatchNorm)
                      VGG_Loss_Landscape.py
results/              figures/ logs/ models/
```

Commands.

```
# Part 1
python codes/part1_cifar/run_best.py --model resnet18 --optimizer sgd --epochs 150
python codes/part1_cifar/run_best.py --model resnet18 --optimizer manual_sgd --
    epochs 150
python codes/part1_cifar/run_ablations.py --group {width,activation,loss,optimizer
    ,optional}
python codes/part1_cifar/run_strong.py --model wrn28_10 --mix 1 --epochs 300 --tag
     wrn_mix300
python codes/part1_cifar/test_optimizers.py     # validates ManualSGD / ManualAdam
python codes/part1_cifar/manual_net.py --selftest && \
python codes/part1_cifar/manual_net.py --epochs 40   # task 5(b) raw-tensor
    ConvNet
python codes/part1_cifar/visualize.py --model wrn28_10 --ckpt wrn_mix300   #
    filters, t-SNE, ...
python codes/part1_cifar/plot_part1.py && python codes/part1_cifar/plot_strong.py
# Part 2
python codes/VGG_BatchNorm/VGG_Loss_Landscape.py --epochs 20
python codes/VGG_BatchNorm/bn_gradient_analysis.py --epochs 3
```